System Verification and Validation Plan for Spectropica

Team #16, DevSquared
Omar Ghoneim
Meena Al Omran
Marko Popovic
Maya Azar
Amirah Abdelrahman

November 9, 2026

Revision History

Date	Version	Notes
November 2nd 2026	1.0	Notes

Contents

1 Symbols, Abbreviations, and Acronyms **iv**

2 General Information **1**
- 2.1 Summary . . . 1
- 2.2 Objectives . . . 2
- 2.3 Custom Documents . . . 5
- 2.4 Relevant Documentation . . . 5

3 Plan **7**
- 3.1 Verification and Validation Team . . . 7
- 3.2 SRS Verification . . . 9
- 3.3 Design Verification . . . 11
- 3.4 Verification and Validation Plan Verification . . . 12
- 3.5 Implementation Verification . . . 15
- 3.6 Automated Testing and Verification Tools . . . 16
- 3.7 Software Validation . . . 18

4 System Tests **20**
- 4.1 Tests for Functional Requirements . . . 20
  - 4.1.1 Accounts & Profiles . . . 20
  - 4.1.2 Care Team & Permissions . . . 30
  - 4.1.3 Gameplay . . . 36
  - 4.1.4 Game Configuration . . . 42
  - 4.1.5 Reward System . . . 48
  - 4.1.6 Learner Data . . . 51
- 4.2 Tests for Nonfunctional Requirements . . . 54
  - 4.2.1 Usability and Performance . . . 54
  - 4.2.2 Accuracy and Efficiency . . . 56
  - 4.2.3 Portability . . . 57
  - 4.2.4 Security . . . 58
  - 4.2.5 Reliability . . . 58
- 4.3 Traceability Between Test Cases and Requirements . . . 59

5 Unit Test Description **61**
- 5.1 Unit Testing Scope . . . 61
- 5.2 Tests for Functional Requirements . . . 62

5.2.1 Module 1 . . . 62
5.2.2 Module 2 . . . 63
5.3 Tests for Nonfunctional Requirements . . . 63
5.3.1 Module ? . . . 63
5.3.2 Module ? . . . 63
5.4 Traceability Between Test Cases and Modules . . . 63

6 Appendix **64**
6.1 Symbolic Parameters . . . 64
6.2 Usability Survey Questions . . . 64

7 Appendix — Generative AI Use Disclosure **68**
7.1 AI Tools Used . . . 68
7.2 How and Where Used . . . 68
7.3 Verification of AI-Generated Content . . . 68

8 Appendix — Reflection **69**

List of Tables

1 Verification and validation team . . . 8
2 Project-specific SRS checks . . . 10
3 Design verification checklist . . . 13
4 VnV plan verification checklist . . . 14
5 Validation activities . . . 18
6 Traceability Between Test Cases and Functional Requirements 60
7 Traceability Between Test Cases and Non-Functional Requirements . . . 61
8 Symbolic Parameters . . . 64
9 CTM usability survey . . . 66
10 Learner proxy observation guide . . . 67

1 Symbols, Abbreviations, and Acronyms

symbol	description
CTM	Care Team Member, an account type with specific authorizations to manage a Learner profile(s)
Primary CTM	a role belonging to only one CTM associated with a Learner's profile with all permissions and controls
FR	Functional Requirement
NFR	Non-functional Requirement
UC	Use Case
SR	Safety Requirement
SRS	Software Requirements Specification

This document describes the plan for verifying and validating Spectropica, a suite of customizable learning games for autistic Learners and the CTMs who support them. The plan builds confidence that Spectropica satisfies the requirements in the SRS, and that those requirements reflect what Learners, CTMs and domain experts need. Section 2 summarizes the software under test, the objectives of verification and validation, and the documents this plan relies on. Section 3 describes the verification and validation team and the approaches used to verify the SRS, the design, this plan and the implementation, along with the automated tools and the validation activities. Section 4 specifies the system tests for the functional and nonfunctional requirements and traces each test to the requirements it covers. Section 5 will describe the unit tests once the detailed design is complete. The appendices give the values of the symbolic parameters used in the tests, and the usability survey.

2 General Information

2.1 Summary

Spectropica is a customizable educational game suite designed to help autistic Learners practise skills through interactive activities. The initial games focus on routine sequencing, functional communication, and short-term memory. Care Team Members (CTMs), including parents, caregivers, and educators, can personalize the games based on each Learner's needs, preferences, and abilities.

The system includes Learner profiles, configurable game settings, reusable Templates, rewards, and a Dashboard for reviewing gameplay results. CTMs can manage Learner profiles, assign games, adjust settings, and control access permissions. Learners can participate in games using features such as instructions, pause and restart controls, and rewards. Spectropica also supports device pairing, gameplay data storage, and synchronization. The system prioritizes iPadOS and Android tablets, with web access also planned.

Spectropica is intended to provide educational practice and activity-specific performance information. It is not designed for clinical diagnosis or to establish broader developmental improvements.

2.2 Objectives

The objective of the Verification and Validation (VnV) Plan is to build confidence that Spectropica meets its documented requirements and is suitable for its intended users. Verification will assess whether the system has been developed according to its specifications, while validation will evaluate whether it meets the needs of Learners and CTMs.

The VnV activities will prioritize the qualities identified as most important in the Software Requirements Specification (SRS) and Hazard Analysis. In particular, the Hazard Analysis prioritizes Learner wellbeing and privacy because failures in these areas could result in significant harm. As a result, particular attention will be given to preventing inappropriate sensory stimulation, unauthorized access to Learner information, incorrect gameplay configurations, and inaccurate or lost performance records.

The primary VnV objectives are outlined below.

Learner Wellbeing, Accessibility, and Usability. Establish confidence that Spectropica provides an accessible, understandable, and appropriate environment for Learners. Verification will examine sensory stimulation, gameplay initiation, flashing restrictions, error messages, and gameplay controls, as specified in SR-4 through SR-7 and SR-11. Validation will include CTM usability testing and Learner-focused evaluations. By default, the supervisor and special-education educators will play-test the games as proxies to evaluate sensory suitability, clarity of instructions, accessibility, and opportunities for independent participation. Direct testing with autistic Learners will only be conducted if recommended by the supervisor and the necessary ethics clearance and participant access are obtained. These activities will support evaluation of NFR-USE-1 through NFR-USE-5, with the limitations of proxy testing clearly documented.

Functional Correctness and Configuration. Verify that the games, Learner profiles, Templates, rewards, and CTM permissions function according to the SRS. Testing will focus on ensuring that CTMs can correctly configure and assign games, that the selected settings are applied during gameplay, and that Learners can interact with the games as intended. Requirements-based testing, design reviews, and implementation checks will be used to identify inconsistencies between expected and actual system behaviour.

Privacy, Security, and Data Integrity. Establish confidence that Learner information and gameplay records are protected against unauthorized access, loss, and corruption. Verification will examine CTM permissions (FR-TEAM-7), security requirements (NFR-SEC-1 through NFR-SEC-3), and the privacy and data-handling commitments in SRS Section 7.3. Particular attention will be given to incomplete-session handling, successful data saves, accurate exports, backups, and deletion procedures, as addressed by SR-8, SR-13, and SR-16 through SR-18. The system's implementation will also be reviewed against the applicable privacy and children's-app requirements identified in the SRS.

Reliability and Recovery. Verify that Spectropica behaves predictably during interrupted gameplay, connectivity loss, and unsuccessful data operations. Testing will assess whether incomplete sessions are handled appropriately, authorized gameplay can continue during connectivity interruptions using available cached settings, and data is only reported as saved when the operation has been confirmed. These objectives are supported by SR-8, SR-9, and SR-13.

Educational Content Suitability. Evaluate whether the content and difficulty levels of the games align with Ontario's Grades 3–5 elementary curriculum, as required by NFR-ACCU-2. Each game's content and difficulty levels will be mapped to relevant curriculum expectations and reviewed with the supervisor, with additional educator input where available. This will help establish that the games are appropriate for their intended educational purpose, rather than only confirming that they function correctly.

CTM Usability and Performance. Evaluate whether CTMs can complete common tasks efficiently and understand the information presented by the system. Usability testing will involve at least MIN_PARTICIPANTS adults in CTM-like roles, such as parents, caregivers, educators, and special-education staff. Participants will complete representative tasks involving account creation, Learner setup, device pairing, CTM permissions, game configuration, and Dashboard interpretation. Their performance, feedback, and survey responses will be used to assess NFR-USE-1 through NFR-USE-4 and NFR-PERF-1. Repeated sessions will also help evaluate how easily CTMs remember how to use the system.

Stakeholder Validation. Validate that Spectropica addresses its intended purpose through supervisor and domain expert feedback, prototype demonstrations, CTM usability sessions, and Learner-focused evaluations. Reviews with the supervisor will assess whether the system meets stakeholder expectations and identify requirements that need improvement. CTM testing will provide direct feedback on configuration, navigation, and progress tracking, while educator and expert proxy evaluations will assess Learner-facing gameplay. Direct Learner participation will remain conditional on supervisor guidance and ethics clearance. Findings from these activities will be documented and used to guide improvements to the system.

All requirements in the SRS are included within the planned academic-year scope, with no requirements intentionally deferred. However, the team recognizes that not all requirements can necessarily be supported by the same type of evidence. Design reviews will assess whether planned components and interfaces are appropriately specified, while implementation and system testing will assess their actual behaviour. Validation results will also distinguish between feedback from intended users and feedback from experts acting as proxies. In particular, proxy evaluations can identify potential usability concerns but cannot independently establish that the 80% Learner participation target in NFR-USE-5 has been achieved. Any requirements that cannot be fully verified or validated will be identified along with the limitations of the available evidence.

Objectives Outside the Scope of VnV

Although all SRS requirements remain within the project scope, the following broader objectives are excluded from VnV:

- **Clinical or long-term developmental effectiveness:** The team will not attempt to establish whether Spectropica produces clinical improvements or long-term developmental outcomes. Doing so would require specialized studies beyond the project's educational purpose and available resources.
- **Exhaustive verification of third-party software and infrastructure:** The team will not independently verify the internal correctness of external components such as the Godot engine, operating systems,

or hosting services. However, Spectropica's integration with these components and its behaviour during relevant failures will still be tested.

- **Compatibility with unsupported platforms:** Verification will focus on the platforms and environments specified for Spectropica. Comprehensive compatibility testing across devices, operating systems, and browsers outside the supported scope is excluded due to the project's time and resource constraints.

These exclusions allow the team to focus its VnV efforts on the correctness, safety, privacy, usability, and educational suitability of Spectropica while remaining realistic about the limitations of an academic-year project.

2.3 Custom Documents

Two custom documents were approved for Spectropica, as identified in the Problem Statement and Goals.

Interview Report - Fall Semester: This document will summarize stakeholder interviews and feedback regarding the needs, preferences, and expectations of Learners and CTMs. It will help the team better understand accessibility considerations, gameplay expectations, and progress-tracking needs. The findings will support validation of the project's requirements and inform design improvements.

Usability Evaluation Report - Winter Semester: This document will present the results of usability evaluations involving CTMs and Learner-focused testing. It will examine how users interact with the prototype, including CTM configuration tasks, Dashboard interpretation, and the accessibility and suitability of Learner-facing games. The report will document observations, participant feedback, identified usability concerns, improvements, and any limitations of the evaluation methods.

Together, these documents will provide evidence that Spectropica has been developed with consideration for its intended users and evaluated against their needs.

2.4 Relevant Documentation

The following documents provide the requirements, design information, and evaluation criteria used to guide Spectropica's verification and validation activities.

Software Requirements Specification (SRS): The SRS serves as the primary reference for verification by defining the system's functional, non-functional, safety, security, and data requirements. These requirements establish the expected behaviour and acceptance criteria against which the implementation will be evaluated. They also provide the basis for requirements traceability and test case development.

Hazard Analysis: The Hazard Analysis identifies potential failures that could affect Learner wellbeing, privacy, system reliability, and data integrity. Its hazards, safety requirements, and recommended mitigations help determine which areas require the greatest attention during verification and validation. It also provides a basis for developing safety-related tests and reviewing how the system responds to potentially harmful conditions.

Problem Statement and Goals: This document describes the problem Spectropica aims to address, its intended users, and the project's goals and success criteria. It supports validation by establishing the purpose of the system and the user needs that the completed software should address. It also identifies the approved custom documents.

Development Plan: The Development Plan describes the team's development process, tools, and implementation approach. It informs the selection of appropriate verification activities, including code reviews, automated testing, integration testing, and other quality assurance practices used throughout development.

Module Guide (MG) and Module Interface Specification (MIS): These design documents will provide information about Spectropica's module structure, responsibilities, interfaces, and interactions. They will support design verification by allowing the team to review whether the proposed architecture satisfies the requirements and whether module interfaces are defined consistently. They will also inform module-level and integration testing.

Interview Report: The Interview Report will document stakeholder feedback gathered during the Fall semester. Its findings will help the team assess whether the system's requirements and design decisions reflect the needs of Learners and CTMs.

Usability Evaluation Report: The Usability Evaluation Report will document the results of the Winter semester usability activities, including CTM testing and Learner-focused evaluations. It will provide evidence for assessing usability requirements, identifying areas for improvement, and explaining any limitations of the validation results.

Together, these documents will support a requirements-based VnV pro-

cess by connecting the project's goals, specifications, design decisions, implementation, and evaluation results.

3 Plan

This section describes how Spectropica and its documentation will be verified and validated. Section 3.1 introduces the verification and validation team and each member's responsibilities. Sections 3.2, 3.3 and 3.4 describe how the SRS, the design and this plan are verified through checklist reviews, walkthroughs and supervisor input. Section 3.5 describes the dynamic and static verification of the implementation, and Section 3.6 describes the automated tools that support it. Section 3.7 describes how the team will confirm, with the supervisor, educators and CTM-like participants, that Spectropica meets the needs of its stakeholders.

3.1 Verification and Validation Team

Table 1 lists the members of the verification and validation team and their responsibilities. Each team member owns a set of tests, reviews and activities. Owning a test means writing it, keeping it up to date with the SRS, and making sure it is run before each demo. It does not mean running it: following Section 3.5, manual system tests are run by a team member who did not implement the feature under test. The supervisor, classmate reviewers and external participants also take part in verification and validation, as described below the table.

External participants. Two groups of participants outside the team take part in validation. The first group is at least `MIN_PARTICIPANTS` adults in roles close to the CTM personas, such as parents, educators and special-education staff, who take part in the CTM usability sessions (V3). The second group is special-education educators who play-test the games as proxies for Learners alongside the supervisor (V4). Sessions with Learners are not planned by default. They are run only if the supervisor advises it and ethics clearance is obtained in time, and the ethics board may limit participants to neurotypical or neurodivergent children of the target age.

Table 1: Verification and validation team

Member	**Role**	**Responsibilities**
Omar Ghoneim	Security and privacy lead	Leads the compliance check in Section 3.7, and presents the code walkthroughs for access control and permissions, and for deletion and retention.
Meena Al Omran	Accounts and platforms lead	Runs the platform verification in Section 3.5, and presents the code walkthrough for pairing and invite codes.
Marko Popovic	Data and synchronization lead	Presents the code walkthroughs for the offline queue and synchronization, session recording, and data export. Coordinates the manual fault injection in Section 3.4.
Maya Azar	Validation lead	Leads the CTM usability sessions (V3), including participant recruitment, and the usability survey. Prepares the ethics application for any sessions involving Learners, and tracks the issues raised by the Rev 0 demo feedback (V2).
Amirah Abdelrahman	Learner experience lead	Leads the inspection of Learner-facing assets in Section 3.5, prepares the observation guide for proxy play-testing (V4), and coordinates the curriculum alignment review (V5).
Project supervisor	Domain expert	Expert in psychology and autism. Completes the SRS task-based review (V1), gives feedback after the Rev 0 demo (V2), play-tests the games as a proxy for Learners (V4), and reviews the curriculum alignment table (V5) and any uncertain wording (V6). Advises the team on whether sessions with Learners should be pursued.
Classmate reviewers	Peer reviewers	Review the SRS, the design, and this plan using the checklists in Sections 3.2, 3.3 and 3.4, and record findings as issues in the project repository.

Review rotation. Every pull request needs an approving review from a member other than its author. Reviews follow a fixed rotation in the order of Table 1: each member's pull requests are reviewed by the next member in the table, and the last member's are reviewed by the first. If the assigned reviewer is unavailable, the following member in the rotation reviews instead. A pull request that touches an area owned by another lead in Table 1 also needs that lead's approval.

3.2 SRS Verification

The SRS is verified in three ways: a checklist review by the team and classmate reviewers, a task-based review with the supervisor, and an automated traceability check. The reviews are applied to the submitted SRS, and their findings feed its next revision. Every finding is recorded as an issue labelled `srs-review` in the project repository, citing the affected requirement or section and the checklist item it fails. Accepted changes are recorded in the SRS revision history.

Checklist review. The team and classmate reviewers review the SRS against the course SRS checklist. Each classmate reviewer files at least five issues. The following checklist items are given particular attention, because they apply directly to Spectropica's SRS:

- literal values are replaced by symbolic constants defined in the SRS appendix;
- nonfunctional requirements are verifiable, with fit criteria where appropriate;
- the SRS is consistent, unambiguous and traceable; and
- rationale is given for non-obvious requirements and scope decisions.

The reviewers also apply the project-specific checks in Table 2, which cover parts of the SRS that the course checklist does not address directly.

Task-based review with the supervisor. The supervisor reviews the SRS in a 60-minute meeting (V1 in Section 3.7). Rather than reading the SRS and giving general comments, the supervisor completes the following tasks with the SRS open, while a team member records their answers:

Table 2: Project-specific SRS checks

ID	Check
S1	Use cases agree with the functional requirements on actors, permissions and outcomes.
S2	The data dictionary (SRS Section 7.2) contains every entity, attribute and value that the functional and safety requirements use.
S3	Every stakeholder requirement (SRS Section **??**) appears in the traceability matrix and traces to at least one functional or nonfunctional requirement.
S4	Every hazard in the Hazard Analysis maps to at least one safety requirement, and no requirement or Learner-facing wording implies a clinical diagnosis or developmental conclusion (SRS Section **??** and ULC Non-Clinical Purpose).

T1. Trace SR-LRN-1 to the functional requirements that support it, and judge whether they would protect a Learner with sensory hypersensitivity.
T2. Find the requirements that prevent a game from startling a Learner, such as sudden audio or flashing content.
T3. Review the default settings and reward approach, and judge whether they are appropriate for the intended Learners.
T4. Identify any requirement or wording that implies a diagnosis or a broader developmental conclusion.
T5. Identify any need of autistic Learners or CTMs that is missing from the stakeholder requirements.

Each answer that points to a missing, incorrect or unclear requirement becomes an issue. The team then discusses with the supervisor which issues to resolve in the next revision of the SRS.

Traceability script. A Python script in the project repository collects every requirement ID defined in the SRS and searches the SRS traceability matrices, the use cases, and the test cases and traceability tables of this plan for each one. It reports every requirement that is not referenced, and every test ID in a traceability table that has no matching test case. The script is run before each submission of the SRS and of this plan, and each gap it

reports becomes an issue. The same script provides check P1 in Section 3.4.

Findings so far. Writing the system tests in Section 4 was used as an early review of the SRS, and surfaced the following defects. These are filed as the first `srs-review` issues.

- **Consistency (S1):** UC8 allows any authorized CTM to delete a Learner profile, while FR-LRN-4 allows only the Primary CTM.
- **Data dictionary (S2):** the outcome attribute of a Game Session has no value for incomplete sessions, which SR-8 requires.
- **Traceability (S3):** SR-LRN-2, SR-CUST-1, SR-CUST-2, SR-CTM-1 and SR-DOM-1 do not appear in the SRS traceability matrix, and the change log described in SRS Section **??** has no corresponding functional requirement.
- **Symbolic constants:** the 30-day grace period and the percentages in NFR-USE-1 to NFR-USE-5 are literal values, and the backup retention period referenced by SR-18 is missing from the SRS appendix.
- **Verifiability:** NFR-EFF-1, NFR-RELY-1 and NFR-PERF-2 have no fit criteria.
- **Ambiguity:** FR-CTM-5 does not state who becomes the Primary CTM when a Primary CTM deletes their account, FR-PLAY-8 does not state where the CTM aborts a game from, FR-RWD-2 does not cover a Learner who already owns every reward item, and SR-12 does not state what happens to a hidden item that is already equipped.

3.3 Design Verification

The design to be verified consists of the Hazard Analysis, the module-level design documents that the team will write after the Software Requirements Specification (SRS) and this plan, and low-fidelity wireframes of the Learner Game Suite and the CTM Dashboard. The design covers the Godot client and the server-side functionality described in the Software Requirements Specification (SRS) Section 2.3. Each review is held when the relevant design material is drafted and before the parts of the implementation that depend on it are started. The checks below are written in terms of the design itself, so they apply regardless of how the module-level documents are structured.

Checklist review. The team will review the module-level design against Table 3. Then the wireframes are checked against the same checklist, and classmate reviewers receive it as well. Findings are recorded as issues labelled `design-review`.

Use-case walkthroughs. The team traces four scenarios through the design on sequence diagrams to find missing responsibilities or interfaces: UC1 including loss of connection (alternative flow 6.1–6.2), UC2 including pairing-code lockout, UC8 including restoration during the grace period, and UC9 including expired and reused invite codes.

Wireframe review with the supervisor. Before the user interface is implemented, the team presents wireframes of the Game Suite, a gameplay screen with the pause, restart, and exit controls, a Learner error message, and the CTM settings screen from UC4. The supervisor is asked whether each screen is uncluttered, whether instructions are unambiguous, and whether anything could overwhelm a Learner (SR-LRN-1 to SR-LRN-3, SR-DOM-3, SR-4 to SR-7, SR-11).

3.4 Verification and Validation Plan Verification

Checklist review. The team and classmate reviewers apply Table 4 before each submission of this plan.

Requirement-coverage check. The traceability script from Section 3.2 is also run on this plan to list requirement IDs that no test references (check P1).

Supervisor input. The supervisor is asked to confirm that the usability survey and the Learner-facing safety tests are appropriate for the intended Learners, and that the participant recruitment plan in Section 3.7 is reasonable.

Manual fault injection. No mutation testing tool is assumed to be available for GDScript, so once the tests are implemented the team will measure test-suite strength by hand. A team member who did not write the code introduces one small fault at a time in a separate branch and runs the full

Table 3: Design verification checklist

ID	Check
D1	Every functional, nonfunctional, and safety requirement is allocated to at least one part of the design, and every part of the design supports at least one requirement.
D2	Each part of the design hides one design decision. Each likely change LC1 to LC8 (the Software Requirements Specification (SRS) Section 12) is confined to a few identifiable parts, so that NFR-MAINT-1 (effort below 1/6 of the original development time) is plausible.
D3	Every CTM action passes through one permission check (FR-TEAM-7, SR-3), and Learner devices store only configurations, inventory, and queued session data (the Software Requirements Specification (SRS) Section 7.3).
D4	Every entity and attribute in the data dictionary (the Software Requirements Specification (SRS) Section 7.2) is mapped to storage. Deleting a CTM account does not delete a Learner profile, and Learner deletion removes all data after the 30-day grace period (FR-LRN-6, SR-18).
D5	Failure handling is designed: offline queue (FR-DATA-8, SR-9), saves confirmed only after verification (SR-13), crashed or aborted sessions flagged incomplete (SR-8), and backup and restore (SR-17).
D6	Encryption in transit and at rest, password hashing, and the rule that imported files are never executed (SR-15) are each assigned to a specific part of the design.
D7	Interfaces between parts of the design use the types in the data dictionary and match how the other parts call them.
D8	The gameplay states in Figure 2 of the Software Requirements Specification (SRS) (loading, ready, playing, paused, reward, load error) appear in the design of the game flow.
D9	Every hazard in the Hazard Analysis has at least one design element that mitigates it.
D10	The design can be built and tested on iPadOS, Android, and web exports of Godot.

Table 4: VnV plan verification checklist

ID	Check
P1	Every functional, nonfunctional, and safety requirement in the Software Requirements Specification (SRS) appears in the traceability table of Section 4.3 or has a named static verification method.
P2	Each system test states its control (manual or automatic), initial state, concrete input data, expected output, derivation, and procedure, so that a team member who did not write it can run it.
P3	Expected outputs are derived from the requirements and use cases, not from what the implementation currently does.
P4	Nonfunctional tests have measurable pass criteria, a sample size, and a survey in the appendix where one is needed (NFR-USE-1 to NFR-USE-5, NFR-PERF-1).
P5	The plan is feasible: the devices, participants, and person-hours it assumes are available to the team.
P6	Terminology matches the Software Requirements Specification (SRS), and the document meets the course template (title page, tables of contents, numbered and captioned tables and figures, revision history, appendices).

suite. The faults target at least ten critical rules: the permission check (FR-TEAM-7), the pairing-code retry limit (FR-DEV-4), the 30-day grace period (FR-LRN-6), incomplete-session handling (SR-8), the hidden-item filter (SR-12), the flash limit (SR-6), the export record count (SR-16), the low defaults (SR-4), the invite-code expiry check (FR-TEAM-1), and the start-cue gate (SR-5). The mutation score is the fraction of faults detected by at least one failing test. Every surviving fault leads to a new or strengthened test.

3.5 Implementation Verification

The implementation will be verified dynamically and statically.

Dynamic verification. The system tests in Section 4 and the unit tests in Section 5 are the primary evidence for functional and nonfunctional requirements. Automated tests run in continuous integration on every pull request (Section 3.6). Manual system tests are run by a team member who did not implement the feature, before the Rev 0 demo and again before the final demo.

Pull-request review. Every pull request needs an approving review from a member other than its author, following the rotation in Section 3.1. The reviewer confirms that the change follows the coding standard, uses static typing, includes or updates tests, contains no credentials, and respects the relevant safety requirements. For example, no audio or animation may start before the Learner's start cue (SR-5), and Learner error messages must be calm and short (SR-7).

Code walkthroughs. The team holds a 60-minute walkthrough for each of these safety- or privacy-critical areas: access control and permissions, pairing and invite codes with lockout, deletion and retention, offline queue and synchronization, session recording, and data export. The author presents the code, at least two other members inspect it against the requirements it supports, and defects become issues. Areas that exist at Rev 0 are walked through before the Rev 0 demo, and the rest before the final demo.

Static analysis. Lint and format checks, strict typing warnings, dependency scanning, and secret scanning run automatically (Section 3.6). A pull

request cannot be merged while these checks fail.

Inspection of Learner-facing assets. Screen captures are recorded to count flashes in every one-second window to check SR-6 (no more than three). The the default settings stored for each game are reviewed for low stimulation, volume, and celebration intensity (SR-4), and every task is checked so that it can be completed with touch or mouse input (SR-10).

Platform verification. The manual platform matrix for NFR-PORT-1 to NFR-PORT-3 is run on physical tablets owned by the team, and iOS and Android simulators or emulators for older operating system versions, and Chrome, Firefox, and Edge.

Out of scope. The Godot engine and third-party libraries are assumed to have been verified by their maintainers. The team tests only how Spectropica uses them.

3.6 Automated Testing and Verification Tools

The team will use three automated tools for the Godot client, all of which run from the command line so that they can be used locally and before a pull request is merged. The commands are documented in the project repository, and every team member runs them before submitting a pull request. A change is ready to merge only when all tests pass, the linter reports no errors, the formatter reports no differences, and Godot reports no warnings.

GUT (Godot Unit Test). GUT is the unit testing framework for GDScript. It is installed as an add-on in the project's `addons` directory. Test scripts live in a `test` directory, extend `GutTest`, and are named `test_*.gd`. Each test function is named `test_<unit>_<condition>_<expected result>`, and a comment above it gives the test ID from Sections 4 and 5 and the requirement ID it supports, so that the traceability tables can be checked against the code. The team will use three GUT features in particular:

- doubles and stubs replace the server connection, so that behaviour such as queuing results when the connection is lost (FR-DATA-8, SR-9) can be tested without a network;

- parameterized tests run the same check over several inputs, such as correct, incorrect, expired, and reused pairing and invite codes (FR-DEV-4, FR-TEAM-1); and
- signal watching confirms that game state changes, such as pausing, restarting, and aborting, emit the expected signals (FR-PLAY-5 to FR-PLAY-7).

gdlint and gdformat. These two command-line tools come from gdtoolkit, and the team will install the release that matches the Godot major version used by the project. `gdlint` checks GDScript files against naming, ordering, and design rules, with the team's chosen settings stored in a `.gdlintrc` file in the repository. `gdformat` reformats files to a single style. Team members run `gdformat` on the files they changed before committing, and the pull-request check runs `gdformat --check` and `gdlint` on the whole project. The check fails on any lint error or any file that is not formatted. A consistent style keeps code reviews focused on behaviour, and the lint rules catch problems such as unused or badly named code before a reviewer has to.

GDScript warnings. Godot's GDScript warnings are configured under *Project Settings* → *Debug* → *GDScript*, and the settings are saved in `project.godot`, so the whole team shares them. Several useful warnings are ignored by default, so the team enables them and turns on the option to treat warnings as errors. The enabled warnings include untyped declarations, unsafe property access, unsafe method access, unsafe casts, unused variables, shadowed variables, integer division, and narrowing conversions. Together these make the team declare types for variables and function signatures, which lets Godot find many type mistakes before the game runs.

Limits of these tools. Passing these checks does not show that Spectropica is correct. Lint, format, and warning checks find only style and type problems, and GUT tests find only the faults they were written to detect. The system tests in Section 4, the reviews and walkthroughs in Section 3.5, and the fault-injection exercise in Section 3.4 supply the rest of the evidence.

3.7 Software Validation

For external data that can be used for validation, Spectropica can be validated via the Ontario elementary curriculum for Grades 3 to 5 (NFR-ACCU-2), the WCAG 2.x flashing criterion (SR-6), PIPEDA, GDPR, and app store rules for children's apps (SRS Section **??**). Table 5 lists the validation activities and the stakeholder requirements from SRS Section **??** that each one addresses.

Table 5: Validation activities

ID	Activity	Participants	Validates
V1	SRS task-based review (Section 3.2; UCs UC1–UC10)	Supervisor	SR-LRN-1–SR-LRN-4; SR-CUST-1–SR-CUST-3; SR-CTM-1–SR-CTM-6; SR-DOM-1–SR-DOM-4
V2	Rev 0 demo to supervisor (UCs UC1, UC2, UC3, UC4, UC5, UC6, UC7, UC9, UC10)	Supervisor	SR-LRN-1–SR-LRN-4; SR-CUST-1–SR-CUST-3; SR-CTM-1–SR-CTM-6; SR-DOM-1–SR-DOM-4
V3	CTM usability sessions and survey (UCs UC2, UC3, UC4, UC7, UC9)	At least 10 adults in CTM-like roles	SR-CTM-5, SR-CTM-6, SR-CUST-1; NFR-USE-1 to NFR-USE-4, NFR-PERF-1
V4	Learner-facing proxy play-testing (UCs UC1, UC6)	Supervisor and special-education educators	SR-LRN-1 to SR-LRN-4, SR-DOM-2, SR-DOM-3; NFR-USE-5
V5	Curriculum alignment review (UCs UC1, UC4, UC5)	Supervisor, and an educator where available	SR-CTM-1, SR-DOM-1, SR-DOM-4; NFR-ACCU-2
V6	Non-clinical wording review (SRS Section 2.1; UCs UC1–UC10)	Team and supervisor	SRS Section **??** and ULC Non-Clinical Purpose

V2: Rev 0 demo to the supervisor. The team will demonstrate Spectropica to the supervisor shortly after the scheduled Rev 0 demo. The demonstration follows five scripted scenarios: (1) a CTM registers (UC7), adds a Learner, and pairs a tablet (UC2); (2) the CTM personalizes settings (UC4) and creates and assigns a Template (UC5); (3) the Learner plays a game, using pause and the guide, and claims a reward (UC1, UC6); (4) the CTM reviews and exports the Learner's data (UC3); and (5) the Primary CTM invites another CTM and edits their permissions (UC9, UC10). Afterward the supervisor completes a feedback form that rates each stakeholder requirement in SRS Section **??** as satisfied, partly satisfied, or not satisfied, with comments. Every partial or unsatisfied rating is turned into an issue labelled `validation`, and the team prioritizes these with the supervisor and resolves them before the final demo. Changes to requirements are recorded in the Software Requirements Specification (SRS) revision history.

V3: CTM usability sessions. At least `MIN_PARTICIPANTS` adults will be recruited in roles close to the CTM personas, such as parents, educators, and special-education staff, through the supervisor and the team's contacts. At least `MIN_PARTICIPANTS` are needed to be able to satisfy the 90% target across NFR-USE-1–NFR-USE-4. Each participant attempts these tasks while an observer records time, errors, and comments: create an account, add and pair a Learner, invite a CTM, personalize a game, and read the dashboard (UC7, UC2, UC9, UC4, and UC3, respectively). Participants then complete the survey in the Appendix. A second session at least one day later repeats the tasks to measure memorability (NFR-USE-3).

V4: Learner-facing validation. Testing with children is research involving human participants and would need ethics clearance, so the team will not run sessions with Learners unless the supervisor advises it and clearance is obtained in time. By default, the supervisor and special-education educators play-test the games in UC1 and UC6 as proxies, using an observation guide that asks whether anything would overstimulate or understimulate a Learner, whether instructions are unambiguous, and whether a Learner could retry freely and feel a sense of accomplishment. Their findings stand in for the Learner measurement in NFR-USE-5, and the report of that measurement states that proxies were used.

V5: Curriculum alignment review. A table is made that maps the content and each difficulty level of every game to Grade 3 to 5 curriculum expectations from the Ontario Ministry of Education. The supervisor reviews the table for appropriateness to the intended Learners (including the game and customization flows in UC1, UC4, and UC5), and educators outside the team review it where available.

V6: Non-clinical wording review. Before each demo, the team checks user-visible text and dashboard labels to confirm that they describe performance inside the games only and do not claim diagnosis or broader developmental improvement (SRS Section **??** and ULC Non-Clinical Purpose). The supervisor reviews any wording the team is unsure about.

Compliance check. Each commitment in SRS Section **??** is mapped to the relevant PIPEDA and GDPR principles and to the app store children's-app rules, and then verifies that the implementation provides the corresponding behaviour.

4 System Tests

The following tests concern the system itself, covering functional and non-functional requirements.

4.1 Tests for Functional Requirements

The following categorization of testing areas is grouped by similarity of behaviours rather than directly correlating with the split in Section 6 of the SRS regarding functional requirements.

4.1.1 Accounts & Profiles

This testing area covers the lifecycle of CTM accounts and Learner profiles with their paired devices. The tests trace to the FR-CTM, FR-LRN and FR-DEV requirements in Section 6 of the SRS.

CTM Accounts

1. test-fr-ctm-1

Control	Manual
Initial State	The application is on the CTM registration page. No account exists for `newctm@example.com`. A verified account already exists for `existingctm@example.com`.
Input	Name: Test CTM; email: `newctm@example.com`. Each of the following is submitted separately, with all other fields valid: (a) a password shorter than `MIN_PASSWORD_LENGTH` (b) a password of valid length that does not meet the SR-1 standard (c) a re-typed password that does not match (d) terms & conditions not accepted (e) the email `existingctm@example.com` in place of `newctm@example.com`
Output	The account is not created and no verification email is sent. The system displays an error stating that account creation failed, with a reason that matches the source of the failure. For case (e), the system also prompts the tester to log in to the existing account.
Derivation	FR-CTM-1 requires registration input to be validated, FR-CTM-2 requires the reason for an error to be displayed, and SR-1 requires password strength to be enforced on account creation. An invalid password, a mismatched password, or not accepting the terms & conditions must therefore block account creation and display a matching reason. UC7 alternative flow 4.2 also specifies that an email already associated with an account leads to a prompt to log in.

Test Procedure	For each case (a)–(e), a tester enters and submits the input on the registration page. The tester confirms the correct error message is shown, checks that no new CTM record with the submitted email exists in the database, and checks that the test inbox received no verification email.

2. test-fr-ctm-2

Control	Manual
Initial State	The application is on the CTM registration page. No account exists for `newctm@example.com`.
Input	Name: Test CTM; email: `newctm@example.com`. Password & re-typed password is valid, terms & conditions are accepted, and email is not already in use. This is followed by acknowledging the verification email, then logging in with the previously entered credentials.
Output	The verification email is acknowledged, the account is created, and the tester is logged in to the account. The system displays a message confirming account creation.
Derivation	FR-CTM-3 says that a verified email is followed by the account being created. FR-CTM-4 specifies that the CTM can log in to their account using the information verified in registration.
Test Procedure	The tester enters the valid registration information outlined in the input. Upon submitting, the tester opens the email's inbox to acknowledge a verification email. The tester then attempts to log in to this account using the registration information.

3. test-fr-ctm-3

Control	Manual
Initial State	CTM A is logged in. CTM A is a non-primary member of Learner L1's care team and owns a template that is assigned to L1. CTM B is the Primary CTM of L1.
Input	CTM A chooses to delete their account. When asked to re-enter their password, CTM A first enters an incorrect password, then the correct password, and confirms the deletion.
Output	With the incorrect password, the deletion is refused and the account is unchanged. With the correct password, the account is scheduled for deletion after `GRACE_PERIOD` and CTM A is logged out. CTM A no longer appears in L1's care team and their template is no longer assigned to L1. L1's profile and data still exist and remain accessible to CTM B.
Derivation	FR-CTM-5 says that a CTM can delete their account with a scheduled grace period, un-linking all Learners and templates. SR-3 requires the password to be re-entered before an account is deleted. SRS Section 11 specifies that deleting a CTM account does not delete a Learner's profile.
Test Procedure	The tester logs in as CTM A and chooses to delete the account, entering an incorrect password first and confirming that the deletion is refused. The tester then repeats the deletion with the correct password. The tester checks that the CTM record has a deletion time set to `GRACE_PERIOD` from now, then logs in as CTM B to confirm that L1's profile and data are unchanged, CTM A is absent from the care

team, and the template is no longer assigned to L1.

4. test-fr-ctm-4

Control	Manual
Initial State	CTM A's account is scheduled for deletion and is still within `GRACE_PERIOD`.
Input	CTM A logs in with their credentials and chooses to restore their account. The test server's clock is then advanced past the original deletion time.
Output	The account is restored and is no longer scheduled for deletion. The account still exists after the original deletion time has passed, and CTM A can log in and use the application normally.
Derivation	FR-CTM-6 says that a CTM can restore their account during the grace period. SRS Section **??** specifies that data restored within the grace period is no longer scheduled for deletion.
Test Procedure	The tester logs in as CTM A and restores the account. The tester checks that the deletion time of the CTM record is cleared, advances the test server's clock past the original deletion time, and confirms that the account still exists by logging in as CTM A again.

5. test-fr-ctm-5

Control	Manual
Initial State	CTM A is logged in and is on the Dashboard.

Input	No interaction with the application for `INACTIVITY_TIMEOUT`, followed by an attempt to open a Learner's settings.
Output	The session has ended. The system returns the tester to the login page and does not display the Learner's settings.
Derivation	SR-2 requires the system to end a CTM's session after a period of inactivity, so no Learner information should be accessible once the session has ended.
Test Procedure	The tester logs in as CTM A, leaves the application idle for `INACTIVITY_TIMEOUT`, then attempts to open a Learner's settings. The tester confirms that the login page is shown and that no Learner information was displayed.

Learner Profiles

1. test-fr-lrn-1

Control	Manual
Initial State	CTM A is logged in and has no Learners on their account.
Input	CTM A chooses to add a Learner, enters the name Test Learner, confirms they are the Learner's legal guardian, and authorizes one game with default settings.
Output	The Learner profile is created under CTM A's account, and CTM A is shown as the Primary CTM of the Learner. The authorized game appears in the Learner's game configuration, and the system displays a pairing code for the Learner's devices.
Derivation	FR-LRN-1 says that a CTM can create a new Learner profile under their account, FR-LRN-2

	designates the creator as the Primary CTM, and FR-LRN-3 allows the CTM to input the Learner's information. UC2 specifies that a pairing code is displayed once the Learner's information is entered.
Test Procedure	The tester logs in as CTM A and completes the steps outlined in the input. The tester confirms that the Learner appears on CTM A's account, checks that the care team membership record marks CTM A as the Primary CTM, and checks that a pairing code is displayed.

2. test-fr-lrn-2

Control	Manual
Initial State	CTM A is logged in and is the Primary CTM of Learner L1. CTM B is a member of L1's care team with permission to view L1's data. L1 is paired to tablet T1 and has recorded game sessions.
Input	CTM A chooses to delete L1's profile, accepts the offer to export L1's data, re-enters their password, and confirms the deletion.
Output	The export contains L1's data. L1's profile is scheduled for deletion after `GRACE_PERIOD`, and CTM A receives a confirmation of the deletion. T1 is unlinked and can no longer open L1's profile. CTM B no longer sees L1 on their account.
Derivation	FR-LRN-4 says that the Primary CTM can delete a Learner's profile with a scheduled grace period, and FR-LRN-5 allows the Learner's data to be exported before deletion. SR-3 requires the password to be re-entered before a Learner profile is deleted. FR-LRN-8 says that

	a Learner cannot access their deleted profile, and UC8 specifies that paired devices are unlinked and other CTMs lose access.
Test Procedure	The tester logs in as CTM A and completes the steps outlined in the input. The tester opens the exported file to confirm it contains L1's data, then checks that L1's record has a deletion time set to `GRACE_PERIOD` from now. The tester confirms that T1 can no longer open L1's profile, and logs in as CTM B to confirm that L1 no longer appears.

3. test-fr-lrn-3

Control	Manual
Initial State	CTM A is logged in and is the Primary CTM of Learner L1. L1's profile is scheduled for deletion and is still within `GRACE_PERIOD`.
Input	CTM A chooses to restore L1's profile. The test server's clock is then advanced past the original deletion time.
Output	L1's profile and recorded game sessions are restored, and the profile is no longer scheduled for deletion. The profile still exists after the original deletion time has passed.
Derivation	FR-LRN-7 says that the Primary CTM can restore a Learner's profile during the grace period. UC8 alternative flow 6.2 specifies that restoring the profile restores the Learner's data and cancels the scheduled deletion.
Test Procedure	The tester logs in as CTM A and restores L1's profile. The tester confirms that L1's game sessions are shown again and that the deletion time of L1's record is cleared, then advances the

test server's clock past the original deletion time and confirms that L1's profile still exists.

4. test-fr-lrn-4

Control	Manual
Initial State	Learner L1's profile is scheduled for deletion and has recorded game sessions, game configurations, inventory items, a paired device and a care team. A backup of the stored data was taken after the deletion was scheduled.
Input	The test server's clock is advanced past GRACE_PERIOD.
Output	No record associated with L1 remains in the database or in any backup, and L1's profile can no longer be restored.
Derivation	FR-LRN-6 says that all Learner-related data is deleted after the grace period. SR-18 requires the system to verify that the deletion has completed and to remove the data from backups. SRS Section **??** specifies that permanent deletion includes all backups.
Test Procedure	The tester advances the test server's clock past GRACE_PERIOD and queries every table holding a Learner identifier for L1's identifier, confirming that no records are returned. The tester repeats the query on the latest backup, then logs in as L1's former Primary CTM to confirm that no option to restore L1 is shown.

Device Pairing

1. test-fr-dev-1

Control	Manual

Initial State	CTM A is the Primary CTM of Learner L1, and a pairing code for L1 is displayed on CTM A's account. Tablets T1 and T2 are unpaired, with the application open on the pairing screen.
Input	The pairing code is entered on T1. CTM A then generates a new pairing code, which is entered on T2.
Output	Each pairing is confirmed on both the device and CTM A's account. T1 and T2 both show L1's Learner screen with the same authorized games, and both devices are listed under L1 on CTM A's account.
Derivation	FR-DEV-1 says that a Learner can pair multiple devices, FR-DEV-2 requires each pairing to be confirmed to both parties, and FR-DEV-3 allows a CTM to generate a new pairing code. UC2 specifies that the Learner can see the correct Learner screen on their paired devices.
Test Procedure	The tester enters the pairing code on T1 and confirms the pairing message on T1 and on CTM A's account. The tester generates a new code from CTM A's account, enters it on T2, and confirms the pairing message again. The tester then compares the Learner screens on T1 and T2 and checks that both devices are listed under L1.

2. test-fr-dev-2

Control	Manual
Initial State	CTM A is the Primary CTM of Learner L1, and a pairing code for L1 is active. Tablet T1 is unpaired, with the application open on the pairing screen.

Input	An incorrect code is entered on T1 PAIRING_RETRY_LIMIT times, followed by the correct code. After PAIRING_LOCKOUT_DURATION has passed, the original code is entered again. CTM A then generates a new code, which is entered on T1.
Output	Once the retry limit is reached, further pairing attempts on T1 are blocked, and the correct code is refused. After the lockout period, the original code is still refused because it has been nullified. The new code pairs T1 successfully.
Derivation	FR-DEV-4 says that the pairing method is nullified after a specific number of retries, and FR-DEV-5 requires further attempts to be blocked for a period of time after the retry limit is reached. FR-DEV-3 allows a CTM to generate a new code, which UC2 alternative flow 6.3 uses to recover from failed attempts.
Test Procedure	The tester enters an incorrect code on T1 until the retry limit is reached, then enters the correct code and confirms that it is refused. The tester advances the test server's clock past PAIRING_LOCKOUT_DURATION, enters the original code again, and confirms that it is still refused. The tester generates a new code from CTM A's account, enters it on T1, and confirms that T1 is paired.

4.1.2 Care Team & Permissions

This testing area covers how CTMs join, are managed within, and leave a Learner's care team, as well as how permissions restrict what each CTM can do. The tests trace to the FR-TEAM requirements in Section 6 of the SRS.

Joining a Care Team

1. test-fr-team-1

Control	Manual
Initial State	CTM A is logged in and is the Primary CTM of Learner L1. CTM C has a verified account and is not part of L1's care team.
Input	CTM A chooses to add a CTM to L1's care team, selects permission to view L1's data only, and shares the generated invite code with CTM C. CTM C enters the invite code on their account.
Output	The system generates an invite code that expires after `INVITE_EXPIRY`. CTM C is added to L1's care team with only the selected permission, L1 appears on CTM C's account, and CTM A is notified that CTM C has joined.
Derivation	FR-TEAM-1 says that the Primary CTM can add other CTMs to the Learner's care team via invite code. UC9 specifies that the invite code has an expiry time, that the new CTM is limited to the permissions selected by the Primary CTM, and that the Primary CTM is notified when the new CTM joins.
Test Procedure	The tester logs in as CTM A and generates an invite with the permission outlined in the input, then checks that the code's expiry is set to `INVITE_EXPIRY` from now. The tester logs in as CTM C, enters the code, and confirms that L1 appears on the account. The tester checks that the care team membership record for CTM C holds only the selected permission, and confirms that CTM A received a notification.

2. test-fr-team-2

Control	Manual

Initial State	CTM A is the Primary CTM of Learner L1. CTM C is logged in and is not part of L1's care team. CTM D has already joined L1's care team using an earlier invite code.
Input	CTM C attempts to join L1's care team in each of the following cases separately: (a) an incorrect code is entered `INVITE_RETRY_LIMIT` times, followed by the correct code (b) the correct code is entered after `INVITE_EXPIRY` has passed (c) the code already used by CTM D is entered (d) the correct code is entered after CTM A has cancelled the invite After each case, CTM A generates a new invite code, which CTM C enters.
Output	In every case, CTM C is not added to L1's care team and the system displays a reason for the failure. In case (a), further attempts are blocked for `INVITE_LOCKOUT_DURATION` once the retry limit is reached, and the correct code is refused. The newly generated code adds CTM C to L1's care team.
Derivation	FR-TEAM-1 says that CTMs join a care team via invite code, FR-TEAM-2 allows the Primary CTM to cancel an outgoing invite, and FR-TEAM-3 allows the Primary CTM to generate a new invite. UC9 alternative flows 7.1–7.3 specify that an incorrect, expired or used code is rejected, that retries are limited before further attempts are blocked, and that the Primary CTM can generate a new code.
Test Procedure	For each case (a)–(d), the tester sets up the code as outlined in the input, advancing the test server's clock past `INVITE_EXPIRY` for case (b), and logs in as CTM C to enter it. The tester

	confirms that a failure reason is shown and that no membership record exists for CTM C on L1. The tester then generates a new code as CTM A, enters it as CTM C, and confirms that CTM C joins L1's care team. Before the next case, CTM C is removed from L1's care team.

Managing a Care Team

1. test-fr-team-3

Control	Manual
Initial State	CTM A is logged in and is the Primary CTM of Learner L1. CTM B is a member of L1's care team with permission to view L1's data and to personalize L1's settings, and is logged in on a separate device with L1's settings open.
Input	CTM A removes CTM B's permission to personalize settings and saves the change. CTM B then attempts to save a change to L1's settings. CTM A then chooses to remove CTM B from L1's care team, re-enters their password, and confirms the removal.
Output	After the permission change, CTM B's attempt to save is refused, and CTM B is notified that their permissions have changed. After the removal, L1 no longer appears on CTM B's account and CTM B is notified of the removal.
Derivation	FR-TEAM-4 says that the Primary CTM can edit the permissions of other CTMs, and UC10 specifies that new permissions are applied immediately and the affected CTM is notified. FR-TEAM-5 says that the Primary CTM can remove another CTM from the care team, and SR-3 requires the password to be re-entered before a CTM is removed.

Test Procedure	The tester logs in as CTM A on one device and as CTM B on another. The tester changes CTM B's permissions as CTM A, then attempts to save a setting as CTM B without reloading, and confirms that the save is refused and the setting is unchanged. The tester then removes CTM B as CTM A, and confirms on CTM B's device that L1 is no longer shown and that both notifications were received.

2. test-fr-team-4

Control	Manual
Initial State	CTM A is the Primary CTM of Learner L1. CTM B is a member of L1's care team.
Input	CTM B requests to transfer ownership of L1's profile to their account. CTM A first declines the request. CTM B then requests the transfer again, and CTM A accepts it and re-enters their password.
Output	When CTM A declines, ownership is unchanged. When CTM A accepts, CTM B becomes the Primary CTM of L1 and CTM A is no longer the Primary CTM. L1 has exactly one Primary CTM.
Derivation	FR-TEAM-6 says that a CTM can transfer ownership of a Learner's profile from another CTM, and UC2 alternative flows 1.1–1.2 specify that the current owner must confirm the transfer. SR-3 requires the password to be re-entered before ownership is transferred. UC10 specifies that there is exactly one Primary CTM for a Learner.
Test Procedure	The tester logs in as CTM B and requests the transfer, then logs in as CTM A and declines it,

confirming that CTM A is still shown as the Primary CTM. The tester repeats the request as CTM B and accepts it as CTM A with the password. The tester confirms that CTM B is shown as the Primary CTM and checks that only one care team membership record for L1 is marked as primary.

3. test-fr-team-5

Control	Manual
Initial State	CTM A is the Primary CTM of Learner L1. CTM B is a member of L1's care team with permission to view L1's data only. CTM C has a verified account and is not part of L1's care team.
Input	CTM B and CTM C each attempt the following actions on L1: (a) change a personalized setting (b) assign a template (c) invite a CTM to the care team (d) delete the profile CTM B attempts each action through the application, and CTM B and CTM C each attempt it by sending the request directly to the server.
Output	Every attempt is refused and L1's data is unchanged. CTM B can still view L1's data.
Derivation	FR-TEAM-7 says that a CTM cannot perform actions they do not explicitly have permission to do. FR-LRN-4 and FR-TEAM-1 also limit deleting a Learner's profile and inviting CTMs to the Primary CTM. Hiding an action in the application does not satisfy FR-TEAM-7 on its own, so the server must also refuse the request.

Test Procedure	For each action (a)–(d), the tester logs in as CTM B and attempts the action through the application, then sends the same request directly to the server using CTM B's session, then using CTM C's session. The tester confirms that every attempt is refused and checks that L1's records in the database are unchanged. The tester then confirms that CTM B can still open L1's data.

4.1.3 Gameplay

This testing area covers a Learner's experience during a game, from selecting a game in the game suite to completing or leaving it, including how the system behaves when the connection is lost or the game fails to load. The tests trace to the FR-PLAY requirements in Section 6 of the SRS, FR-DATA-7 and FR-DATA-8, and the safety requirements SR-5, SR-7, SR-8, SR-9 and SR-11. The tests follow UC1 and the gameplay states in Figure 2 of the SRS.

Playing a Game

1. test-fr-play-1

Control	Manual
Initial State	Learner L1 is paired to tablet T1. L1 is authorized to play game G1 with personalized settings and is not authorized to play game G2. The application on T1 is on the game suite.
Input	The Learner selects G1, gives the start cue, completes every task, and exits to the game suite from the completion screen.
Output	The game suite shows G1 and does not show G2. G1 loads with L1's personalized settings and the completion screen is shown after the final task. The session is recorded as completed, with the settings used and the scores of each task.

Derivation	FR-PLAY-1 says that a Learner cannot play games they are not explicitly authorized to play, and FR-PERS-4 says that the personalized settings are loaded when a Learner plays a game. FR-DATA-7 requires the Learner's performance in the session to be saved. UC1 specifies the flow from selecting a game to exiting it.
Test Procedure	The tester confirms that only G1 appears in the game suite on T1, then plays G1 to completion as outlined in the input. The tester checks that the settings shown in the game match L1's personalized settings, then checks that the session record for L1 has a completed outcome, the settings used, and a score for each task.

2. test-fr-play-2

Control	Manual
Initial State	Learner L1 is paired to tablet T1 and is authorized to play game G1, with music and sound effects set above zero. The device volume on T1 is turned up.
Input	The Learner selects G1 and waits without giving the start cue. The Learner then gives the start cue and attempts to move to the next task before completing the current one, then completes the current task.
Output	Before the start cue, the game does not start and no audio or animation is played. After the start cue, the game begins. The next task is not presented while the current task is incomplete, and is presented once it is completed.
Derivation	FR-PLAY-2 says that a game does not start until the Learner gives a cue that they are

ready, and SR-5 requires that no game audio or animation begins before this cue. FR-PLAY-3 says that the next task is only presented once the current task is completed.

Test Procedure The tester records the screen and audio of T1 while selecting G1 and waiting without giving the start cue, then gives the cue and attempts to skip ahead before completing the first task. The tester reviews the recording to confirm that there is no audio or animation before the start cue, and confirms that the second task appeared only after the first was completed.

3. test-fr-play-3

Control Manual

Initial State Learner L1 is paired to tablet T1 and is playing game G1. CTM A, the Primary CTM of L1, has permitted the guide and playing again for G1.

Input Each of the following is performed separately during a game of G1:

(a) the Learner pauses the game, then resumes it
(b) the Learner restarts the game
(c) the Learner aborts the game
(d) the Learner opens the guide for the current instruction, then closes it
(e) CTM A aborts the game from T1
(f) the Learner completes the game, chooses to play again, then completes it and returns to the game suite
(g) cases (d) and (f) are repeated after CTM A has disallowed the guide and playing again

Output Pause, restart and exit controls are visible and responsive throughout. In case (a), the game stops and resumes on the same task. In case (b), the game starts again from the first task and the earlier session is recorded as restarted. In cases (c) and (e), the Learner returns to the game suite and the session is recorded as incomplete. In case (d), the guide for the current instruction is shown and closing it returns to the same task. In case (f), a new session starts and the Learner returns to the game suite afterward. In case (g), the guide and play again options are not available.

Derivation FR-PLAY-4 allows the Learner to read a guide if permitted by the CTM, FR-PLAY-5 to FR-PLAY-7 allow the Learner to pause, restart and abort a game, and FR-PLAY-8 allows the CTM to abort a Learner's game. FR-PLAY-9 allows the Learner to play again if permitted by the CTM, and FR-PLAY-10 allows the Learner to return to the game suite. SR-11 requires the pause, restart and exit controls to be visible and responsive at all times, and SR-8 requires aborted sessions to be recorded as incomplete.

Test Procedure For each case (a)–(g), the tester starts G1 on T1 and performs the actions outlined in the input, confirming that the pause, restart and exit controls remain visible and respond throughout. The tester confirms the behaviour described in the output for each case, then checks the outcome recorded for each session in the database.

4. test-fr-play-4

Control Manual

Initial State	Learner L1 is paired to tablet T1 and is authorized to play game G1, with L1's settings already stored on T1. T1 is connected to the network.
Input	Each of the following is performed separately: (a) T1's network connection is turned off after the start cue, the Learner completes G1, and the connection is turned back on (b) T1's network connection is turned off before the application is opened, the Learner plays G1 to completion, and the connection is turned back on (c) the application is force-closed during G1, then reopened with the network connected
Output	In cases (a) and (b), the game plays without interruption using the settings stored on T1. The session results are kept on T1 until the connection returns, are then uploaded and recorded as completed, and are removed from T1's local storage. In case (c), the session is recorded as incomplete.
Derivation	SR-9 requires that a loss of connection does not interrupt a session in progress or prevent a Learner from playing authorized games with settings already stored on the device. FR-DATA-8 requires performance to be queued locally for future syncing without a stable connection, and SRS Section **??** specifies that queued data is deleted from local storage after syncing. SR-8 requires sessions that end through a crash to be recorded as incomplete.
Test Procedure	For cases (a) and (b), the tester plays G1 as outlined in the input and confirms that the game is not interrupted. While the connection is off, the tester checks that no new session record exists in the database. After

	reconnecting, the tester checks that the session record appears with a completed outcome and that T1's local queue is empty. For case (c), the tester force-closes the application, reopens it, and checks that the session is recorded as incomplete.

5. test-fr-play-5

Control	Manual
Initial State	Learner L1 is paired to tablet T1 and is authorized to play game G1. A test build is installed on T1 in which G1 is set to fail when loading.
Input	The Learner selects G1 from the game suite, then selects the action shown in the error message.
Output	The system displays a short, calm error message with no alarming sounds or visuals, offering a single action to return to the game suite. Selecting the action returns the Learner to the game suite.
Derivation	SR-7 requires error messages shown to a Learner to be calm and short, to avoid alarming sounds or visuals, and to offer a single action to return to the game suite. UC1 alternative flow 2.1 specifies that a game may fail to load with the correct settings.
Test Procedure	The tester records the screen and audio of T1 while selecting G1. The tester confirms that one error message with a single action is shown, reviews the recording for alarming sounds or visuals, and selects the action to confirm that it returns to the game suite.

4.1.4 Game Configuration

This testing area covers how CTMs tailor games for a Learner, through game and difficulty authorizations, personalized settings, and templates. The tests trace to the FR-PERS and FR-TMPL requirements in Section 6 of the SRS.

Personalization

1. test-fr-cfg-1

Control	Manual
Initial State	CTM A is logged in and is the Primary CTM of Learner L1. L1 is paired to tablet T1, is authorized to play game G1 at every difficulty, and is not authorized to play game G2.
Input	CTM A performs each of the following in order: (a) authorizes G2 for L1 (b) removes the authorization of G1 for L1 (c) makes only one difficulty of G2 available to L1
Output	In case (a), G2 appears in L1's game suite on T1. In case (b), G1 no longer appears in L1's game suite and cannot be launched. In case (c), only the selected difficulty of G2 can be chosen on T1.
Derivation	FR-PERS-1 says that a CTM can toggle the authorization of specific games for a Learner, and FR-PLAY-1 says that a Learner cannot play games they are not explicitly authorized to play. FR-PERS-2 says that an authorized CTM can toggle the authorization of specific difficulties of a game for a Learner.
Test Procedure	For each case (a)–(c), the tester makes the change as CTM A, then checks the game suite on T1. For case (b), the tester also attempts to launch G1 from a game suite screen that was

	open before the change, and confirms that the game does not start. For case (c), the tester opens G2 on T1 and confirms that only the selected difficulty is available.

2. test-fr-cfg-2

Control	Manual
Initial State	CTM A is logged in and is the Primary CTM of Learner L1, which was just created and has never had its settings changed. L1 is paired to tablet T1 and is authorized to play game G1.
Input	CTM A opens L1's settings for G1. CTM A changes the stimulation level, reward, prompt level, volume levels and celebration intensity, and saves the settings. The Learner then plays G1 on T1. CTM A then makes the server unreachable from their device, changes the stimulation level again, and saves.
Output	Before any changes, the stimulation level, volume levels and celebration intensity are set to their low default values. The first save is confirmed, and the session on T1 uses the new settings. The second save is not confirmed, the CTM is informed that it failed, and the previous settings are kept.
Derivation	SR-4 requires the default stimulation level, volume and celebration intensity to be low until a CTM changes them. FR-PERS-3 says that an authorized CTM can choose personalized settings of a game for a Learner, and FR-PERS-4 says that these settings are loaded when the Learner plays the game. SR-13 requires a save to be confirmed only after it has been verified, and the previous values to be

	kept if saving fails. UC4 specifies the settings that can be personalized.
Test Procedure	The tester logs in as CTM A, opens L1's settings for G1, and confirms that the stimulation level, volume levels and celebration intensity match the low defaults. The tester makes and saves the changes outlined in the input, plays G1 on T1, and checks that the settings recorded for the session match the saved settings. The tester then disconnects CTM A's device from the server, makes and saves another change, and confirms that a failure message is shown and that L1's stored settings are unchanged.

Templates

1. test-fr-tmpl-1

Control	Manual
Initial State	CTM A is logged in and is the Primary CTM of Learners L1 and L2. L1 has personalized settings for game G1 and no assigned template. L2 already has another template assigned for G1. L1 is paired to tablet T1.
Input	CTM A creates template TP1 from G1, adds, changes and removes one content item each, changes the colour palette and volume levels, chooses a difficulty, and saves TP1 in a new folder named Folder 1. CTM A then assigns TP1 to L1 and L2, reviews the preview, confirms the assignment, and confirms that L2's existing template should be replaced. The Learner then plays G1 on T1.
Output	TP1 is saved in Folder 1 with the changed content, settings and difficulty. Before the

	assignment, the system shows a preview of TP1 and lists L1 and L2 along with what will be replaced for each, and requires confirmation. The system also asks whether to replace L2's existing template. After confirmation, TP1 is assigned to both Learners. The session on T1 uses TP1's content, settings and difficulty instead of L1's personalized settings.
Derivation	FR-TMPL-1 says that a CTM can create a template from an existing game, FR-TMPL-3 to FR-TMPL-5 allow the CTM to change the template's content, settings and difficulty, and FR-TMPL-9 allows templates to be organized in folders. FR-TMPL-6 says that assigning a template to a Learner overrides any previous settings, and FR-PERS-4 says that personalized settings are not loaded when a template is assigned. SR-14 requires the affected Learners, what will be replaced, and a preview to be shown before assignment, and UC5 alternative flow 6.1 specifies that the CTM is asked before an existing template is replaced.
Test Procedure	The tester logs in as CTM A and creates TP1 as outlined in the input, then confirms that TP1 appears in Folder 1 with the changes made. The tester assigns TP1 to L1 and L2, confirming that the preview, the list of affected Learners and the replacement prompt for L2 are shown before the assignment takes effect. The tester then plays G1 on T1, confirms that TP1's content and difficulty are presented, and checks that the session record references TP1 and the settings used match TP1's settings.

2. test-fr-tmpl-2

Control	Manual
Initial State	CTM A is logged in and is the Primary CTM of Learners L1 and L2. Template TP1 for game G1 is assigned to both Learners, and each Learner had different personalized settings for G1 before the assignment. L1 is paired to tablet T1 and L2 is paired to tablet T2.
Input	CTM A un-assigns TP1 from L1, reviews the preview, and confirms. The Learner then plays G1 on T1. CTM A then deletes TP1, reviews the preview, and confirms. The Learner then plays G1 on T2.
Output	Before each change, the system shows a preview of TP1, the affected Learners and what will be replaced, and requires confirmation. After the un-assignment, the session on T1 uses L1's previous personalized settings. After the deletion, TP1 no longer appears in CTM A's templates and the session on T2 uses L2's previous personalized settings.
Derivation	FR-TMPL-7 says that un-assigning a template from a Learner restores their previous settings, and FR-TMPL-8 says that deleting a template un-assigns it from any associated Learners. SR-14 requires the affected Learners, what will be replaced, and a preview to be shown before a template is un-assigned or deleted.
Test Procedure	The tester records L1's and L2's personalized settings for G1 before the test. The tester un-assigns TP1 from L1 as CTM A, confirming that the preview and affected Learners are shown, then plays G1 on T1 and checks that the settings recorded for the session match L1's earlier settings exactly. The tester then deletes TP1 as CTM A in the same way, confirms that TP1 is no longer listed, plays G1 on T2, and

checks that the settings recorded for the session match L2's earlier settings exactly.

3. test-fr-tmpl-3

Control	Manual
Initial State	CTM A owns template TP1. CTM C is logged in and has no templates. Two test files are prepared: a truncated template file, and a template file with a script placed in a content field that would write a marker file if executed.
Input	Each of the following is performed separately: (a) CTM A exports TP1, and CTM C imports the exported file (b) CTM C imports the truncated template file (c) CTM C imports the template file containing a script
Output	In case (a), CTM C has a template with the same content, settings and difficulty as TP1. In cases (b) and (c), the import is rejected with a reason and no template is created. In case (c), the script is not executed.
Derivation	FR-TMPL-2 says that a CTM can import a template from another CTM, and UC5 specifies that a template can be exported to a file. SR-15 requires imported files to be treated as untrusted data and never executed.
Test Procedure	For case (a), the tester exports TP1 as CTM A, imports the file as CTM C, and compares the two templates' content, settings and difficulty. For cases (b) and (c), the tester imports each file as CTM C, confirms that a failure reason is shown, and checks that no template was added to CTM C's account. For case (c), the tester also checks that the marker file was not created on the device or the server.

4.1.5 Reward System

This testing area covers how a Learner earns rewards from games and unlocks, equips and views cosmetic items, as well as how CTMs limit which items are available. The tests trace to the FR-RWD requirements in Section 6 of the SRS.

Earning Rewards

1. test-fr-rwd-1

Control	Manual
Initial State	Learner L1 is paired to tablet T1 and is authorized to play game G1. L1 owns every item in G1's reward pool except item I1.
Input	The Learner completes G1 and claims the reward. The Learner then starts G1 again and aborts it.
Output	After the completed game, the reward offered is I1, and claiming it adds I1 to L1's inventory as a game reward. After the aborted game, no reward is offered and L1's inventory is unchanged.
Derivation	FR-RWD-1 says that a Learner can claim a reward after a completed game, and UC1 specifies that the reward is claimed after the completion screen. FR-RWD-2 says that duplicate items are not offered as rewards, so the only item that can be offered is the one L1 does not own.
Test Procedure	The tester plays G1 to completion on T1, confirms that I1 is the reward offered, and claims it. The tester checks that L1's inventory records contain I1 with a game reward source. The tester then starts G1 again, aborts it, and confirms that no reward is offered and that L1's inventory records are unchanged.

Cosmetic Items

1. test-fr-rwd-2

Control	Manual
Initial State	Learner L1 is paired to tablets T1 and T2 and has a known currency balance. Item I2 costs less than the balance, item I3 costs more than the balance, and item I4 is in the same category as I3 and costs less than the balance. L1 owns item I5, which is not equipped. The application on T1 is on the items screen.
Input	Each of the following is performed separately on T1: (a) the Learner buys I2 (b) the Learner equips I5 (c) the Learner attempts to unlock I3 (d) the Learner equips I2 while the server is unreachable
Output	In case (a), I2 is added to L1's inventory and the cost is deducted from the balance. In case (b), I5 is shown as equipped on T1, and also on T2. In case (c), I3 is not unlocked, and the system displays the reason, the conditions for unlocking I3, and I4 as an alternative. In case (d), the save fails, I5 remains equipped, and the Learner is informed with a calm, short message.
Derivation	FR-RWD-3 allows a Learner to view cosmetic items, FR-RWD-4 allows a Learner to unlock an item they meet the conditions for, and FR-RWD-5 allows a Learner to equip an item they own. FR-RWD-6 and FR-RWD-7 require a reason, the unlock conditions, and buyable alternatives to be shown when an item cannot be unlocked. SR-13 requires the previous values to be kept and the Learner informed if saving

	fails, and SR-7 requires error messages shown to a Learner to be calm and short. UC6 specifies that currency is deducted when an item is bought.
Test Procedure	For each case (a)–(d), the tester performs the action on T1 as outlined in the input. For case (a), the tester checks L1's inventory records and balance. For case (b), the tester confirms that I5 is shown as equipped on T1 and on T2. For case (c), the tester confirms the reason, conditions and alternative shown. For case (d), the tester disconnects T1 from the server before equipping I2, confirms the message shown, and checks that I5 is still recorded as equipped.

2. test-fr-rwd-3

Control	Manual
Initial State	CTM A is logged in and is the Primary CTM of Learner L1. L1 is paired to tablets T1 and T2, owns item I5 and has it equipped. L1 does not own item I6 and has enough currency to buy it.
Input	CTM A hides I5 and I6 from L1's items screen. The Learner then opens the items screen on T1 and on T2. Requests to buy I6 and to equip I5 are then sent directly to the server using L1's device session.
Output	I5 and I6 do not appear on the items screen on either tablet, and I5 is no longer equipped. The direct requests are refused, and L1's inventory and balance are unchanged.
Derivation	SR-12 requires cosmetic items hidden by a CTM to be hidden from the items screen and not to be purchasable or equippable by the Learner. UC6 specifies that the items screen applies any filters set by the CTM.

Test Procedure	The tester hides I5 and I6 as CTM A, then opens the items screen on T1 and on T2 and confirms that neither item appears and that I5 is not shown as equipped. The tester sends the buy and equip requests directly to the server using L1's device session, confirms that both are refused, and checks that L1's inventory records and balance are unchanged.

4.1.6 Learner Data

This testing area covers how CTMs view, filter and export a Learner's recorded game data, and how access to that data is restricted. The tests trace to the FR-DATA requirements in Section 6 of the SRS and to the safety requirements SR-8 and SR-16.

Viewing and Exporting Data

1. test-fr-data-1

Control	Manual
Initial State	CTM A is logged in and is the Primary CTM of Learner L1. L1 has a seeded history of 10 sessions of game G1 and 4 sessions of game G2 between October 1 and October 20. The G1 sessions include sessions on October 6 and October 15, and two aborted sessions. The expected averages and statistics are computed by hand from the seeded scores before the test.
Input	CTM A opens L1's data and expands the game history. CTM A then filters the history to October 6 – October 15, opens the data for G1, and switches between each available data representation.
Output	The main page shows L1's most recent sessions and the average performance for each game,

	matching the hand-computed values. The expanded history lists all 14 sessions. The filtered history lists exactly the sessions between October 6 and October 15, including those on both dates. The G1 data shows L1's statistics and session history, and every representation shows the same values. The aborted sessions appear in the history as incomplete and are not counted in the averages.
Derivation	FR-DATA-1 allows an authorized CTM to view a Learner's general data, FR-DATA-2 requires an expandable overview of the game history, and FR-DATA-3 allows the history to be viewed between specific dates. FR-DATA-4 and FR-DATA-5 allow game-specific data to be viewed in multiple representations. SR-8 requires incomplete sessions not to be counted in a Learner's performance averages. UC3 specifies the flow for viewing a Learner's data.
Test Procedure	The tester loads the seeded history and logs in as CTM A. The tester compares the main page, the expanded history and the filtered history against the seeded sessions and the hand-computed values. The tester then opens the G1 data, switches between each representation, and confirms that each one shows the same values and that the aborted sessions are marked incomplete and left out of the averages.

2. test-fr-data-2

Control	Manual
Initial State	CTM A is logged in and is the Primary CTM of Learner L1, which has the seeded history from test-data-1.

Input	CTM A selects G1 and the dates October 6 – October 15 on the dashboard, then exports L1's data for this selection.
Output	The export contains every G1 session record for L1 between October 6 and October 15 and no other records. The record count reported by the export matches the count shown on the dashboard for the same selection.
Derivation	FR-DATA-6 allows an authorized CTM to export a Learner's game data. SR-16 requires an export to contain every record for the selected Learner, game and date range, and to report a record count that matches the dashboard for the same selection.
Test Procedure	The tester notes the record count shown on the dashboard for the selection, then exports the data. The tester opens the exported file, compares its records against the seeded G1 sessions between October 6 and October 15, and confirms that the reported count matches the dashboard count.

3. test-fr-data-3

Control	Manual
Initial State	Learner L1 has the seeded history from test-data-1. CTM C is logged in and is not part of L1's care team.
Input	CTM C looks for L1 on their account. Requests to view and export L1's data are then sent directly to the server using CTM C's session and L1's identifier.
Output	L1 does not appear on CTM C's account. The direct requests are refused and no Learner data is returned.

Derivation	FR-DATA-1 and FR-DATA-6 limit viewing and exporting a Learner's data to authorized CTMs, and FR-TEAM-7 says that a CTM cannot perform actions they do not explicitly have permission to do.
Test Procedure	The tester logs in as CTM C and confirms that L1 does not appear on the account. The tester then sends the view and export requests directly to the server using CTM C's session, and confirms that both are refused and that the responses contain none of L1's data.

4.2 Tests for Nonfunctional Requirements

The following subsections map to the different categories of nonfunctional requirements in section 8 of the SRS, with some combined with each other due to overlap in testing strategies. Note that the carrying out of tests requiring a Learner's participation will be left to the discretion of the project supervisor and may be omitted if there are ethical or safety concerns.

4.2.1 Usability and Performance

1. test-nfr-use-1

   Type: Functional, Manual, Dynamic

   Initial State: Software is running on a compatible device, with no account logged in.

   Input/Condition: At least 10 individuals in the age range of a potential CTM (20-50) go through the account setup process, inputting their information and proceeding through the steps. Participants complete a usability survey afterwards.

   Output/Result: Accounts are created with the individuals' information. At least 80% of participants report an average score of at least `MIN_SURVEY_SCORE` for ease of use and satisfaction with performance.

   How test will be performed: In-person sessions will be scheduled with participants. The developers will facilitate the test with their personal

device and administer the survey afterwards on the same device through a Google form.

2. test-nfr-use-2

   Type: Functional, Manual, Dynamic

   Initial State: Software is running on a compatible device, with a sample CTM account signed in and a sample Learner device paired.

   Input/Condition: At least 10 individuals in the age range of a potential CTM (20-50) are given a series of actions to complete on the software such as navigating to a view or changing a setting. Participants will be asked to do each action three times total. Participants will complete a usability survey afterwards.

   Output/Result: The participants are able to complete the specified actions. There is a 50% increase of speed in completing these actions when doing so for the third time. At least 80% of participants report an average score of at least MIN_SURVEY_SCORE for ease in doing and learning these actions.

   How test will be performed: In-person sessions will be scheduled with participants. The developers will facilitate the test with their personal devices, time how long the participants take to complete actions, and administer the survey afterwards on the same device through a Google form.

3. test-nfr-use-3

   Type: Functional, Manual, Dynamic

   Initial State: Software is running on a compatible device, with a sample CTM account signed in and a sample Learner device paired.

   Input/Condition: At least 10 CTM's for an autistic child (referred to as the Learner) in the intended range of 8-10 facilitates playing one game. Both CTM and Learner complete a usability survey afterwards.

   Output/Result: The game is completed. At least 80% of CTM participants report an average score of at least MIN_SURVEY_SCORE for ease in facilitating gameplay. At least 80% of Learner participants report an

average score of at least MIN_SURVEY_SCORE for ease of participating in gameplay (the CTM may answer on their behalf).

How test will be performed: In-person sessions will be scheduled with participants. The developers will facilitate the test with their personal devices and administer the surveys afterwards on the same device through Google forms.

4.2.2 Accuracy and Efficiency

1. test-nfr-acc-1

   Type: Functional, Manual, Dynamic

   Initial State: Software is running on a compatible device, with a sample CTM account signed in and a sample Learner device paired.

   Input/Condition: 1-2 developers will simulate gameplay data for a chosen game, recording the sessions.

   Output/Result: The data view for the game contains these sessions' data. The data is visualized in this view across at least 2 metrics. Observations of trends made by 2 developers who did not participate in simulating data are aligned with the gameplay sessions.

   How test will be performed: 3-4 developers will carry out the test in-person using 2 of their personal devices (one for the CTM account and one for the Learner device). The developers will spend 15 minutes reviewing the data to measure whether data is indeed quickly understood through visualizations.

2. test-nfr-acc-2

   Type: Non-functional, Manual, Static

   Initial State: The contents of the games implemented within the scope of capstone are summarized in a document by the developers.

   How test will be performed: 5 educators in Ontario who have taught or are currently teaching students in intended grade target of 3-5 will review the document outlined above within a week.

   Result: The participants will confirm that the games' educational aspects are aligned with Ontario's curriculum for grades 3-5.

4.2.3 Portability

1. test-nfr-port-1

   Type: Non-functional, Manual, Dynamic

   Initial State: An iPad running iPadOS 27 is available for developers' use.

   Input/Condition: The software will be installed on the device.

   Output/Result: The software is successfully installed and ran on the device.

   How test will be performed: 1 developer will carry through the test on the device.

2. test-nfr-port-2

   Type: Non-functional, Manual, Dynamic

   Initial State: An Android Tablet running Android 17 is available for developers' use.

   Input/Condition: The software will be installed on the device.

   Output/Result: The software is successfully installed and ran on the device.

   How test will be performed: 1 developer will carry out the test on the device.

3. test-nfr-port-3

   Type: Non-functional, Manual, Dynamic

   Initial State: A laptop is available for developers' use; will likely be a personal laptop.

   Input/Condition: Running the software will be attempted on the following browsers: Google Chrome, Mozilla Firefox, and Microsoft Edge.

   Output/Result: The software successfully runs on the browsers.

   How test will be performed: 1 developer will carry out the test on the laptop.

Note that older operating systems will be considered out of scope for testing due to lack of access.

4.2.4 Security

1. test-nfr-sec-1

   Type: Non-functional, Manual, Static

   How test will be performed: 2 developers who did not work on the account section of the software will do a code walkthrough of the processes for creating and logging into an account, as well as how the associated information is stored.

   Output/Result: The developers confirm that the privacy and data retention requirements outlined in section 7.3 of the SRS; specifically regarding what information is stored and how for CTM's and Learners.

2. test-nfr-sec-2

   Type: Functional, Manual, Dynamic

   Initial State: The software is running on a compatible device, with a sample CTM account set up and a sample Learner device linked. Another sample CTM account is also set up and linked to the Learner.

   Input/Condition: From the second CTM's account, some settings for the Learner device are changed.

   Output/Result: From the Primary CTM's account, these actions are recorded and visible in the log referenced in the SRS.

   How test will be performed: 2 developers will carry out the test in-person using their personal devices to simulate the scenario.

4.2.5 Reliability

1. test-nfr-rely-1

   Type: Non-functional, Automatic, Dynamic

Initial State: At least one game has been implemented in the software.

Input/Condition: Response times within gameplay are recorded and stored.

Output/Result: Response times are consistent with each other, with a standard deviation of +-0.5 seconds.

How test will be performed: These measures are written into the implementation code and stored, being accessible to the developers through export in table form.

Note: NFR-ACCESS-1 and NFR-ACCESS-2 do not need to be tested as they will implicitly be implemented. NFR-MAINT-1 cannot be tested as it requires implementing a major change.

4.3 Traceability Between Test Cases and Requirements

Test Case	Functional and Safety Requirements
test-fr-ctm-1	FR-CTM-1, FR-CTM-2, SR-1
test-fr-ctm-2	FR-CTM-3, FR-CTM-4
test-fr-ctm-3	FR-CTM-5, SR-3
test-fr-ctm-4	FR-CTM-6
test-fr-ctm-5	SR-2
test-fr-lrn-1	FR-LRN-1, FR-LRN-2, FR-LRN-3
test-fr-lrn-2	FR-LRN-4, FR-LRN-5, FR-LRN-8, SR-3
test-fr-lrn-3	FR-LRN-7
test-fr-lrn-4	FR-LRN-6, SR-18
test-fr-dev-1	FR-DEV-1, FR-DEV-2, FR-DEV-3
test-fr-dev-2	FR-DEV-3, FR-DEV-4, FR-DEV-5
test-fr-team-1	FR-TEAM-1
test-fr-team-2	FR-TEAM-1, FR-TEAM-2, FR-TEAM-3
test-fr-team-3	FR-TEAM-4, FR-TEAM-5, SR-3
test-fr-team-4	FR-TEAM-6, SR-3

test-fr-team-5	FR-TEAM-1, FR-TEAM-7, FR-LRN-4
test-fr-play-1	FR-PLAY-1, FR-PERS-4, FR-DATA-7
test-fr-play-2	FR-PLAY-2, FR-PLAY-3, SR-5
test-fr-play-3	FR-PLAY-4, FR-PLAY-5, FR-PLAY-6, FR-PLAY-7, FR-PLAY-8, FR-PLAY-9, FR-PLAY-10, SR-8, SR-11
test-fr-play-4	FR-DATA-8, SR-8, SR-9
test-fr-play-5	SR-7
test-fr-cfg-1	FR-PERS-1, FR-PERS-2, FR-PLAY-1
test-fr-cfg-2	FR-PERS-3, FR-PERS-4, SR-4, SR-13
test-fr-tmpl-1	FR-TMPL-1, FR-TMPL-3, FR-TMPL-4, FR-TMPL-5, FR-TMPL-6, FR-TMPL-9, FR-PERS-4, SR-14
test-fr-tmpl-2	FR-TMPL-7, FR-TMPL-8, SR-14
test-fr-tmpl-3	FR-TMPL-2, SR-15
test-fr-rwd-1	FR-RWD-1, FR-RWD-2
test-fr-rwd-2	FR-RWD-3, FR-RWD-4, FR-RWD-5, FR-RWD-6, FR-RWD-7, SR-7, SR-13
test-fr-rwd-3	SR-12
test-fr-data-1	FR-DATA-1, FR-DATA-2, FR-DATA-3, FR-DATA-4, FR-DATA-5, SR-8
test-fr-data-2	FR-DATA-6, SR-16
test-fr-data-3	FR-DATA-1, FR-DATA-6, FR-TEAM-7

Table 6: Traceability Between Test Cases and Functional Requirements

Test Case	Non-Functional Requirements
test-nfr-use-1	NFR-USE-1, NFR-PERF-1
test-nfr-use-2	NFR-USE-2, NFR-USE-3
test-nfr-use-3	NFR-USE-4, NFR-PERF-2
test-nfr-acc-1	NFR-EFF-1, NFR-ACCU-1

test-nfr-acc-2	NFR-ACCU-2
test-nfr-port-1	NFR-PORT-1
test-nfr-port-2	NFR-PORT-2
test-nfr-port-3	NFR-PORT-3
test-nfr-sec-1	NFR-SEC-1, NFR-SEC-2
test-nfr-sec-2	NFR-SEC-1, NFR-SEC-2
test-nfr-rely-1	NFR-RELY-1
test-fr-tmpl-1	NFR-ACCESS-3
test-fr-play-3	NFR-ACCESS-4

Table 7: Traceability Between Test Cases and Non-Functional Requirements

5 Unit Test Description

[This section should not be filled in until after the MIS (detailed design document) has been completed. —SS]

[Reference your MIS (detailed design document) and explain your overall philosophy for test case selection. —SS]

[To save space and time, it may be an option to provide less detail in this section. For the unit tests you can potentially layout your testing strategy here. That is, you can explain how tests will be selected for each module. For instance, your test building approach could be test cases for each access program, including one test for normal behaviour and as many tests as needed for edge cases. Rather than create the details of the input and output here, you could point to the unit testing code. For this to work, you code needs to be well-documented, with meaningful names for all of the tests. —SS]

5.1 Unit Testing Scope

[What modules are outside of the scope. If there are modules that are developed by someone else, then you would say here if you aren't planning on verifying them. There may also be modules that are part of your software, but have a lower priority for verification than others. If this is the case, explain your rationale for the ranking of module importance. —SS]

5.2 Tests for Functional Requirements

[Most of the verification will be through automated unit testing. If appropriate specific modules can be verified by a non-testing based technique. That can also be documented in this section. —SS]

5.2.1 Module 1

[Include a blurb here to explain why the subsections below cover the module. References to the MIS would be good. You will want tests from a black box perspective and from a white box perspective. Explain to the reader how the tests were selected. —SS]

1. test-id1

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

2. test-id2

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

3. ...

5.2.2 Module 2

...

5.3 Tests for Nonfunctional Requirements

5.3.1 Module ?

1. test-id1

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input/Condition:

   Output/Result:

   How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input:

   Output:

   How test will be performed:

5.3.2 Module ?

...

5.4 Traceability Between Test Cases and Modules

[Provide evidence that all modules have been considered. —SS]

References

6 Appendix

Additional Information.

6.1 Symbolic Parameters

Parameter	Value
MIN_PARTICIPANTS	10 participants
MIN_SURVEY_SCORE	8/10
MIN_PASSWORD_LENGTH	12 characters
INACTIVITY_TIMEOUT	30 minutes
GRACE_PERIOD	30 days
PAIRING_RETRY_LIMIT	5 attempts
PAIRING_LOCKOUT_DURATION	15 minutes
INVITE_EXPIRY	48 hours
INVITE_RETRY_LIMIT	5 attempts
INVITE_LOCKOUT_DURATION	15 minutes

Table 8: Symbolic Parameters

6.2 Usability Survey Questions

The following survey is completed by participants in the CTM usability sessions (V3 in Section 3.7), through a Google Form on the facilitating developer's device at the end of each session. No identifying information is collected. Rated questions use a scale from 1 (very difficult or very unsatisfied)

to 10 (very easy or very satisfied). A rating of at least MIN_SURVEY_SCORE counts as experiencing little to no difficulty for the nonfunctional requirement that the question measures.

Observer record. During each session, an observer records the following for each task (creating an account, adding and pairing a Learner, inviting a CTM, personalizing a game, and reading the dashboard): the time taken, the number of errors, whether the participant needed help, and any comments. The times from the first and second sessions are compared to measure memorability (NFR-USE-3), alongside Q16.

The following guide is used by the supervisor and special-education educators when play-testing each game as proxies for Learners (V4 in Section 3.7). The guide is completed once per game. Rated questions use the same 1–10 scale as the usability survey, and a rating of at least MIN_SURVEY_SCORE counts as a Learner being easily able to participate for NFR-USE-5. If sessions with Learners are approved by the ethics board, the same guide is completed by an observer during those sessions, and the report states which kind of participant each result comes from.

Table 9: CTM usability survey

ID	Question	Response	Measures
Background			
Q1	Which best describes your role: parent or guardian, educator, special-education staff, or other?	Choice	–
Q2	How comfortable are you using tablets and computer applications?	1–10	–
Q3	Have you supported an autistic child's learning before?	Yes / No	–
Setup			
Q4	How easy was it to create your account?	1–10	NFR-USE-1
Q5	How easy was it to add a Learner and pair their device?	1–10	NFR-USE-1
Q6	How easy was it to invite another CTM to the Learner's care team?	1–10	NFR-USE-1
Navigation			
Q7	How easy was it to find a Learner's settings?	1–10	NFR-USE-2
Q8	How easy was it to find where to start a game for a Learner?	1–10	NFR-USE-2
Q9	How easy was it to personalize a game's settings for a Learner?	1–10	NFR-USE-2
Facilitating gameplay			
Q10	How easy was it to set up and start a game for a Learner?	1–10	NFR-USE-4
Q11	How confident would you feel supervising a Learner during a game, including pausing, stopping or helping them?	1–10	NFR-USE-4
Dashboard and response time			
Q12	How easy was it to understand the Learner's performance from the dashboard?	1–10	NFR-EFF-1
Q13	How satisfied were you with how quickly the application responded when logging in and opening a Learner's data?	1–10	NFR-PERF-1
Open feedback			
Q14	What, if anything, was confusing or frustrating?	Free text	–
Q15	What would you change about the application?	Free text	–

Table 10: Learner proxy observation guide

ID	Question	Response	Measures
O1	Could any visual, sound or animation overstimulate a Learner with sensory hypersensitivity? If so, where?	Yes / No, with notes	SR-LRN-1, SR-DOM-3
O2	Could the game understimulate a Learner with sensory hyposensitivity? If so, where?	Yes / No, with notes	SR-LRN-1, SR-DOM-3
O3	How clear are the rules and instructions for a Learner who may not read independently?	1–10	SR-LRN-2
O4	Could a Learner take their time and retry without feeling pressured?	Yes / No, with notes	SR-LRN-3
O5	How likely is the game to give a Learner a sense of accomplishment?	1–10	SR-LRN-4
O6	How likely is the game to hold a Learner's attention?	1–10	SR-DOM-2
O7	How easily could a Learner participate in the game without frustration or confusion?	1–10	NFR-USE-5
O8	Any other observations about the game's suitability for autistic Learners.	Free text	–

7 Appendix — Generative AI Use Disclosure

This appendix describes the artificial intelligence (AI) tools used during document preparation, their uses, and how the team reviewed the resulting edits.

7.1 AI Tools Used

The team used ChatGPT (OpenAI, GPT-6) to assist with document preparation and proofreading.

7.2 How and Where Used

ChatGPT assisted with LaTeX formatting, grammar, spelling, and clarity edits to existing text throughout the VnV Plan. The team developed all technical content, requirements, and testing approaches; the tools were used only to format and refine the wording of that content.

7.3 Verification of AI-Generated Content

The team reviewed all AI-assisted formatting and language edits before including them in the final document. The review checked that edits preserved the original meaning and that technical information, requirement references, and testing details remained accurate and consistent with the project's documentation. The team takes full responsibility for the accuracy, completeness, consistency, and quality of the submitted VnV Plan.

8 Appendix — Reflection

The following responses reflect on preparing the verification and validation (VnV) Plan and the skills needed to carry out the planned activities.

1. **What went well while writing this deliverable?**

   One aspect that went well while working on the VnV Plan was completing the deliverable ahead of schedule. This gave our team additional time to peer review each other's sections, provide feedback, and make revisions before submission. Having this extra time allowed us to identify areas that needed clarification and ensure that the different sections were consistent with one another. Another positive aspect was the opportunity to meet with our supervisor and domain expert, who provided valuable feedback that we were able to incorporate into the deliverable. Her insight helped us better understand the needs of our intended users and consider important factors when developing our verification and validation plans. Overall, completing the deliverable early and incorporating feedback from both our team members and supervisor helped us strengthen the quality of our work.

2. **What pain points did you experience during this deliverable, and how did you resolve them?**

   One of the main challenges we encountered was deciding which testing methods and test cases would be most appropriate for Spectropica. Since our system includes several different features and user interactions, it was difficult to determine how to evaluate everything thoroughly while keeping our testing plans realistic and achievable within the project timeline.

   To address this, we focused on identifying the most important features of our system and prioritizing the testing activities associated with them. We also considered the feedback provided by our supervisor to help guide our decisions. This process helped us recognize that an effective VnV Plan is not simply about including as many tests as possible, but about selecting meaningful tests that address the system's requirements and most important risks.

3. **What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation**

of the project?

Amirah — Test Planning and Requirements-Based Verification

One skill I would like to further develop is designing effective test cases and selecting appropriate verification methods based on software requirements. Although I have experience working with software requirements and testing concepts, applying them to a larger project with multiple interconnected features requires additional consideration. Developing this skill will help me identify which requirements need particular attention, establish clear expected outcomes, and ensure that our testing activities provide meaningful evidence of whether Spectropica meets its intended objectives.

Maya — Usability Test Design

I would like to improve my ability to turn usability requirements into clear tasks and measurable results. Spectropica's SRS includes goals such as CTMs being able to configure Learner profiles with little difficulty, but evaluating this requires defining what counts as difficulty and how it will be recorded. Learning to design task-based usability evaluations would help me choose appropriate observations and questions, avoid leading participants, and distinguish problems with the interface from differences in users' experience.

Meena — System and Integration Testing

Although I have a lot of experience from both university and co-op in writing unit tests and doing smoke testing, I haven't done a lot of larger-scale tests like integration tests and system tests. It will be new to have to test, both manually and through automation, that the larger components fit together. I also usually did not have to think about usability in my co-op. I was always just carrying out the specifications made by the UI/UX team. I will have to think critically about this aspect, especially while designing for children.

Marko — Godot Specific Testing

I already have experience and knowledge with Godot's systems, but as for testing and validation I have only used basic debugging tools and relying on players to catch bugs. I have yet to work on something

that requires this much care in regards to the amount of important requirements and things that require validation.

Omar — Full Stack Testing

I have experience building full stack web applications, but I have never used a game engine to export or import data and interact with databases in real time. Debugging a frontend-backend issue has been a skill I've developed over time, but across a game engine with highly sensitive information, it is definitely a new level of security and privacy management I will acquire over the course of the project.

4. **For each knowledge area or skill identified in Question 3, what are at least two ways to acquire it? Which approach will each team member pursue, and why?**

   Amirah — Test Planning and Requirements-Based Verification

   Two ways I could further develop this skill are reviewing software testing resources and examples of requirements-based test plans, or gaining hands-on experience by developing test cases directly from Spectropica's SRS.

   My preferred approach would be to work directly with our project's requirements and develop test cases based on the functionality and expected behaviour described in the SRS. This would allow me to apply testing concepts to our actual system rather than learning them only in theory. I could also use relevant course materials and examples as references when I encounter unfamiliar testing methods. I believe this approach would be the most beneficial because it would help me strengthen my understanding of verification and validation while directly contributing to our project's testing efforts.

   Maya — Usability Test Design

   Two approaches I could take are studying usability-testing guides and example evaluations, or developing a small evaluation for one Spectropica workflow and practising it with a teammate.

   I would choose the practical approach, supported by relevant guides. For example, I could write a task for adding and pairing a Learner, define what successful completion looks like, and record errors, requests

for help, and participant feedback during a trial run. Reviewing the trial would help me identify unclear instructions or measurements before using the procedure with intended users. A teammate's trial would help improve the evaluation method, but would not replace testing with CTMs or Learners.

Meena — Research and Studying

To learn more about system and integration testing, I will have to do some research - especially regarding the specific tech stack that we are using. There is most likely documentation available online, and I think it would also help a lot to study open-source projects that are using our same technologies. I can also review my notes from SFWRENG 3SO3, in which we studied all kinds of testing. For usability, we are studying this in our SFWRENG 3HC3 course (human-computer interfaces). So that course can be a helpful frame of referenes for what kinds of considerations I need to make.

Marko — Godot Specific Testing

One approach I could use is researching the best practices for GUT and integrating it with the rest of the tech stack. I also know that for different types of games, testing comes in different forms. I can try and figure out specific testing approaches that other similar games use and see if they can be implemented into ours. Another approach in tandem with the first would be to create a small practice project on the side with a similar tech stack and situation to Spectropica, so as to gain actual experience and skill.

Omar — Full Stack Testing

The approach in this case is to read about how other systems handle sensitive data, perhaps in the medical field. I am also taking a Data Privacy elective next semester that will aid me in understanding conventions that are applicable to this project.

Regarding design, I will join groups and read forums regarding how game developers are able to debug across a wide range of tools without getting lost trying to find a bug across several platforms. It is fundamentally similar to the full stack debugging in web development, but the structure and process likely has conventions that I do not know of.